

Introduction to Function Pointers

Directing the Hive Flow

In C, functions are not just blocks of code; they are stored in memory at a specific address. A Function Pointer is a variable that stores this memory address.

Unlike a typical pointer that points to data (like an int or a char), a function pointer points to executable code. This allows developers to treat functions as data, passing them around or choosing which one to execute at runtime.

At Codehive, we use function pointers to achieve 'Dynamic Dispatching'—the ability to change program behavior without rewriting the core logic.

Variable Pointers vs. Function Pointers

Navigating Codehive Memory

Variable Pointer: Stores the address of a value (e.g., `int *p = &x;`). It is used to manipulate data.

Function Pointer: Stores the entry point of a function. When you 'dereference' it, the CPU jumps to that address and begins executing instructions.

Key Distinction: You cannot perform pointer arithmetic (like `p++`) on a function pointer, as functions are not stored in contiguous arrays like elements of a list.

The Syntax Breakdown

The Codehive Signature Rule

Declaring a function pointer requires matching the signature of the target function exactly.

Syntax: `return_type (*ptr_name)(parameter_types);`

Importance of Parentheses: The parentheses around `(*ptr_name)` are mandatory. Without them, C interprets the line as a function declaration that returns a pointer (e.g., `int *p(int);` is a function returning an int pointer).

Basic Implementation

Codehive Laboratory Example

Here is how we link a pointer to a simple addition function:

```
#include <stdio.h>

int add(int a, int b) {
    return a + b;
}

int main() {
    // 1. Declare and Initialize
    int (*fp)(int, int) = add;

    // 2. Call via Pointer
    int result = fp(5, 3);

    printf("Result: %d", result);
    return 0;
}
```

Initialization Nuances

Labels and Addresses

In C, the name of a function acts as its address. Therefore, the following two initializations are identical in the Codehive standard:

1. `int (*fp)(int) = &myFunction;` (Explicit address)
2. `int (*fp)(int) = myFunction;` (Implicit address)

Similarly, calling the function can be done as `fp(5)` or `(*fp)(5)`. Modern C compilers treat both as valid instructions to execute the code at that address.

Function Pointers as Callbacks

Enabling Modular Plug-ins

The most powerful use of function pointers is the 'Callback'. This is where you pass a function pointer as an argument to another function.

This allows you to write a generic function (like a 'Sorter') that can use any 'Comparison' function provided by the user.

Codehive Modular Design: By using callbacks, we can update a specific mathematical logic without ever changing the main processing engine.

Arrays of Function Pointers

Building Codehive Menu Systems

You can create an array of function pointers to handle multiple related operations (like a calculator or a navigation menu).

```
int (*ops[])(int, int) = {add, sub, mul, div};  
  
// Execute multiplication dynamically  
int result = ops[2](10, 5);
```

This approach eliminates messy 'switch-case' blocks and makes the program structure much cleaner and faster.

Typedef for Simplified Code

Codehive Readability Standards

Function pointer syntax can become messy. Codehive developers use `typedef` to create alias names for pointer types.

```
typedef int (*MathFunc)(int, int);  
  
// Now we can declare pointers easily  
MathFunc f1 = add;  
MathFunc f2 = sub;
```

This makes function signatures much easier to read when passing them between different modules.

Real-World App: qsort

Standard Library Integration

The C standard library function `qsort()` uses function pointers to sort any type of data.

You provide a comparison function that tells `qsort` how to order two elements. This makes `qsort` the ultimate generic sorting tool.

Without function pointers, the C library would need a different sorting function for every possible data type (ints, doubles, structs, etc.).

Summary & Exam Guide

Final Technical Review

- Purpose: Stores address of a function for dynamic calls.
- Syntax Requirement: `*` and parentheses are essential.
- Flexibility: Enables callbacks, menus, and generic algorithms.
- Memory: Points to the .text/code segment of the program.

One-Line Codehive Definition:

A function pointer in C is a pointer that stores the address of a function instead of a variable.